HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering



DEEP LEARNING AND ITS APPLICATIONS

From Linear Models to Modern Sequence Models

A Comparative Study for Image Classification

Instructor: Le Thanh Sach, Ph.D.
Course: CO3133 – Semester 261
Group: MobyDick

Student Name	Student ID	Contribution
Le Quoc Binh	2352117	Pipeline, CNN/ViT, extensions
Quach Kien Long	2352698	Data/EDA, Linear/MLP models
Le Thanh Lam	2252421	LSTM/GRU & robustness
To Nha Vy	2353360	Error analysis, XAI, report/pages

Ho Chi Minh City, October 2026

Contents

1 Introduction **2**

2 Datasets and Preprocessing **2**

2.1 Dataset Description 2

2.2 Exploratory Data Analysis (EDA) 3

2.3 Data Partitioning and Leakage Prevention 3

2.4 Preprocessing 4

3 Methodology **4**

3.1 Pipeline Overview 4

3.2 Model Architectures 5

4 Experimental Protocol **8**

4.1 Training Configuration and Checkpoint Selection 8

4.2 Experimental Design 9

4.3 Hardware and Software Environment 10

4.4 Reproducibility and Measurement Protocol 10

5 Results and Discussion **11**

5.1 Training Dynamics 11

5.2 Fashion-MNIST Benchmark 11

5.3 Matched Parameter Budget Comparison 14

5.4 CIFAR-10 Extension Benchmark 14

5.5 Robustness and Interpretability 15

5.6 Error Analysis 16

5.7 Limitations 17

6 Conclusion **17**

1 Introduction

Recognizing garments and objects motivates the study of how architectural assumptions affect image classifiers. This report compares image-classification architectures under a shared experimental protocol, using Fashion-MNIST as the primary benchmark and CIFAR-10 as an extension. The objective is to examine predictive performance jointly with parameter requirements, execution time, and robustness, rather than to rank models by accuracy alone.

Image classification assigns a semantic category to an image represented as a discrete pixel grid. Given an input tensor $X \in \mathbb{R}^{C\times H\times W}$, where C denotes the number of color channels and H and W denote image height and width, respectively, the objective is to learn a parameterized mapping $f_\theta : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^K$. This mapping produces unnormalized logits $z = f_\theta(X)$ for $K = 10$ mutually exclusive classes. The corresponding class probabilities are obtained through the softmax function:

$$\hat{y}_k = P(y = k \mid X; \theta) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \quad k \in \{0, \ldots, K-1\} \tag{1}$$

The model parameters θ are estimated by minimizing the multiclass cross-entropy loss over N training samples:

$$\mathcal{L}_{\mathrm{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log \left(\frac{\exp(z_{i,y_i})}{\sum_{j=1}^{K} \exp(z_{i,j})} \right) \tag{2}$$

Main Difficulties: Although Fashion-MNIST depicts low-resolution grayscale garments against uncluttered backgrounds, substantial within-class variation and visual overlap between classes complicate discrimination. The similarity among *Shirt*, *T-shirt/top*, *Pullover*, and *Coat* is particularly challenging for models without local spatial feature extraction. CIFAR-10 introduces additional variation through background clutter, object viewpoint, and illumination in natural RGB images.

2 Datasets and Preprocessing

2.1 Dataset Description

The dataset usage protocol distinguishes pipeline verification, primary comparison, and extension evaluation as follows:

- **Debugging Dataset:** No separate debugging dataset was used. MNIST was excluded from the reported results, and pipeline verification was performed directly on small Fashion-MNIST batches.
- **Primary Comparison Dataset: Fashion-MNIST** (Zalando Research, MIT License) provides $70,000$ grayscale garment images of size $1 \times 28 \times 28$, distributed across 10 classes, and constitutes the primary benchmark.
- **Optional Extension Dataset: CIFAR-10** (Krizhevsky et al., 2009) provides $60,000$ natural color images of size $3 \times 32 \times 32$ across 10 object categories and is used to extend the comparison.

The official dataset sources are the Fashion-MNIST repository and the CIFAR dataset page. Fashion-MNIST consists of labeled Zalando product images and is distributed as gzip-compressed IDX image and label files under the repository's MIT license. CIFAR-10 is a labeled subset of the Tiny Images collection; its official Python distribution uses pickled image batches and label lists. Both datasets are obtained through Torchvision's standard download interface and stored locally. The available records do not establish the exact archival footprint or checksums, and do not document CIFAR-10 reuse terms.

2.2 Exploratory Data Analysis (EDA)

Exploratory data analysis examines input dimensionality, class distributions, and class balance to establish the basis for architectural comparison. Fashion-MNIST is represented by single-channel tensors of shape $(1, 28, 28)$, comprising 784 pixels per image. CIFAR-10 is represented by RGB tensors of shape $(3, 32, 32)$, comprising $3,072$ values per image.

The official Fashion-MNIST training set comprises $60,000$ images, with exactly $6,000$ samples per class; the $10,000$-image test set contains $1,000$ samples per class (Table 1). This uniform distribution (imbalance ratio $= 1.0$) supports the use of unweighted cross-entropy without class-frequency correction. Accuracy and Macro-F1 are both reported to characterize predictive performance and class-specific precision–recall trade-offs.

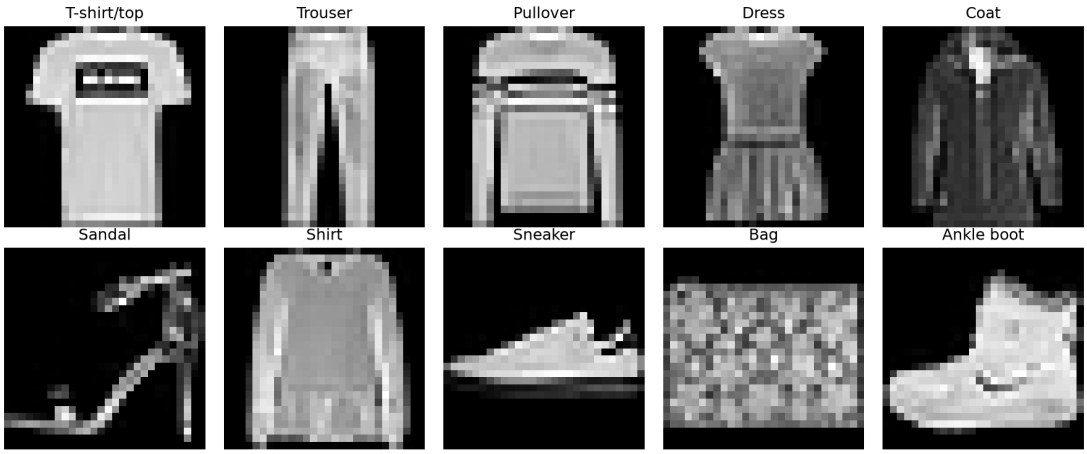


Figure 1: Representative training samples across all 10 classes of Fashion-MNIST.

2.3 Data Partitioning and Leakage Prevention

All architectures within a dataset share the same mini-batch loading procedure and data partitions. The official training set is partitioned at image level by seeded random sampling (seed $= 42$), with split sizes reported in Table 1. Partitioning precedes model initialization, ensuring that the same train and validation subsets are reused throughout the main architecture comparison. The official test split is held separately. Within each experimental run, clean-test benchmark metrics are computed after restoring the checkpoint selected by validation Macro-F1; robustness is assessed separately using perturbed test images.

Partitioning is random and unstratified, without label-based balancing, group-level separation, or additional subset filtering. Training examples are shuffled using a separately seeded pseudorandom generator; validation and test examples retain their dataset order. The training sampler's random state advances across sequential model runs rather than being reset for each architecture, so identical partitions do not imply identical per-epoch mini-batch orders. The global initialization seed is reset before each model is constructed.

Table 1: **Dataset Splitting Protocol and Sample Counts** (Seed = 42).

Dataset	Purpose	Train	Validation	Test
Fashion-MNIST	Primary Benchmark	50,000	10,000	10,000
CIFAR-10[a]	Optional Extension	45,000	5,000	10,000

[a] For CIFAR-10, the official 50,000 training images are split into 45,000 train/5,000 validation with the same fixed seed (42); the official 10,000-image test set is held out.

2.4 Preprocessing

8-bit unsigned integer pixel intensities in $[0, 255]$ are converted into floating-point tensors in $[0.0, 1.0]$. Each channel is centered using a mean of 0.5 and scaled by a standard deviation of 0.5, mapping values to $[-1.0, 1.0]$. For CIFAR-10, these statistics are applied independently to color channels. Training batches have shape $(64, 1, 28, 28)$ for Fashion-MNIST and $(64, 3, 32, 32)$ for CIFAR-10. No image cleaning, resizing, padding, augmentation, or annotation-repair procedure is applied; images and labels are used as supplied by the Torchvision datasets.

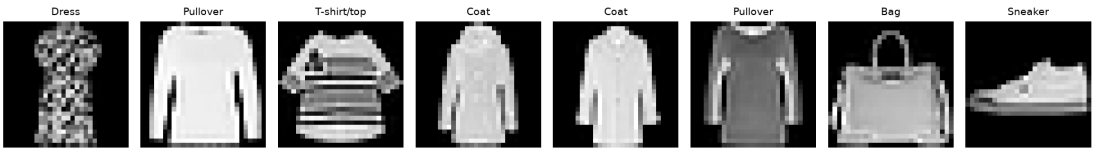


Figure 2: Example batch slice (8 samples) after preprocessing and tensor normalization.

3 Methodology

3.1 Pipeline Overview

Figure 3 summarizes the end-to-end PyTorch pipeline. The architectures are constructed from standard PyTorch layers without pretrained backbones. Torchvision provides dataset loading and preprocessing, while PyTorch supports automatic differentiation, recurrent and Transformer operations, and optimization.

Macro-F1 and confusion matrices are computed using scikit-learn; Matplotlib and NumPy support visualization and numerical aggregation. Predicted labels are obtained by selecting the largest output logit, without additional post-processing. An archived implementation accompanies the study to support reconstruction of the experimental procedure.

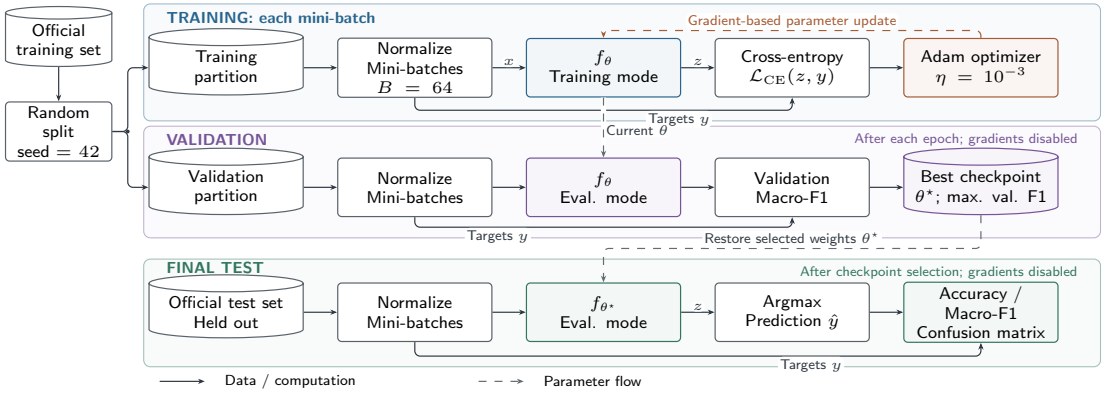


Figure 3: End-to-end PyTorch training, validation checkpointing, and test evaluation pipeline.

3.2 Model Architectures

3.2.1 Linear/Softmax Classifier

The Linear classifier operates directly on flattened image intensities, with the transformation from input to class logits specified in Table 2. It computes $z = Wx + b$, where $W \in \mathbb{R}^{10\times784}$ and $b \in \mathbb{R}^{10}$, and contains $7,850$ trainable parameters.

Table 2: **Linear/Softmax Classifier Architecture** on Fashion-MNIST.

Stage	**Operation**	**Configuration**	**Output**[a]
Vectorization	Flatten	$X \in \mathbb{R}^{1\times28\times28}$	$x \in \mathbb{R}^{784}$
Readout	Linear	$784 \to 10$	10

[a] Operations are listed in execution order, with the batch dimension omitted. The readout produces unnormalized class logits; softmax is not a separate architectural layer.

The cross-entropy objective receives logits directly, without a preceding softmax operation. Its formulation combines log-softmax normalization and negative log-likelihood for numerical stability. The classifier offers rapid inference and directly interpretable weights. Its inductive bias restricts the representation to affine combinations of position-specific pixel intensities and linear decision hyperplanes; spatial invariance is not encoded in the architecture.

3.2.2 Multilayer Perceptron

The MLP operates on the flattened input $x \in \mathbb{R}^{784}$ through two hidden layers and a linear classification head, as specified in Table 3. The architecture contains $235,658$ trainable parameters. ReLU ($\max(0, x)$) introduces piecewise linear nonlinearity while avoiding gradient saturation. Batch normalization normalizes hidden activations to accelerate convergence, whereas dropout ($p = 0.2$) randomly suppresses activations during training to limit feature co-adaptation.

Table 3: **Multilayer Perceptron Architecture** on Fashion-MNIST.

Stage	Operation	Configuration	Output[a]
Hidden 1	Linear	$784 \to 256$	256
	BatchNorm1d	256 features	256
	ReLU	—	256
	Dropout	$p = 0.2$	256
Hidden 2	Linear	$256 \to 128$	128
	ReLU	—	128
	Dropout	$p = 0.2$	128
Readout	Linear	$128 \to 10$	10

[a] Operations are listed in execution order. Output denotes the feature-vector width, with the batch dimension omitted. The readout produces unnormalized class logits.

3.2.3 Convolutional Neural Network

The CNN preserves the 2D input grid $(1, 28, 28)$ and extracts spatial features through two convolutional blocks before applying a fully connected classifier. Table 4 specifies the operations, channel transformations, and intermediate representations.

Table 4: **Convolutional Neural Network Architecture** on Fashion-MNIST.

Stage	Operation	Configuration	Output[a]
Block 1	Conv2d	$1 \to 16$, $k = 3$, $p = 1$	—
	BatchNorm2d	—	—
	ReLU	—	—
	MaxPool2d	$(2, 2)$	$(16, 14, 14)$
Block 2	Conv2d	$16 \to 32$, $k = 3$, $p = 1$	—
	BatchNorm2d	—	—
	ReLU	—	—
	MaxPool2d	$(2, 2)$	$(32, 7, 7)$
Vectorization	Flatten	—	1,568
Classifier	Linear	$1568 \to 128$	128
	ReLU	—	—
	Dropout	$p = 0.3$	—
	Linear	$128 \to 10$	10

[a] Operations are listed in execution order. Convolutional settings k and p denote kernel size and padding; pooling settings specify kernel size and stride. Selected output shapes omit the batch dimension, and dashes indicate omitted intermediate shapes or inapplicable settings. The final output comprises unnormalized class logits.

The complete architecture contains $207,018$ trainable parameters. Shared convolutional kernels extract local edges and textures across spatial locations, while deeper feature maps represent hierarchical combinations of object parts. Max pooling reduces spatial resolution by $2\times$, contributing local tolerance to translation. The flattened representation contains $1,568$ features, which are mapped to class logits by the classification head.

3.2.4 Recurrent Neural Networks: LSTM and GRU

The recurrent architectures interpret each image (C, H, W) as an ordered sequence of horizontal rows. At timestep $t \in \{1, \ldots, 28\}$ on Fashion-MNIST, the input vector $x_t \in \mathbb{R}^{C \cdot W}$ contains 28 features; the corresponding width for CIFAR-10 is 96. The recurrence $h_t = \mathrm{RNN}(x_t, h_{t-1})$ processes the image from top to bottom, imposing a directional, row-sequential inductive bias rather than treating the image as an unordered collection of pixels. Gated state updates support the propagation of information across rows.

The shared processing sequence is specified in Table 5. LSTM and GRU differ in their recurrent gating mechanisms, while retaining the same sequence representation and classification head. Their parameter counts are $214,538$ and $161,290$, respectively.

Table 5: **LSTM and GRU Architectures** on Fashion-MNIST.

Stage	**Operation**	**Configuration**	**Output**[a]
Representation	Row sequencing	$T = H = 28$, $C \cdot W = 28$	$(28, 28)$
Sequence encoder	LSTM or GRU	2 layers, $d_h = 128$; interlayer dropout $p = 0.2$	$(28, 128)$
State selection	Final timestep	$h_{28} \in \mathbb{R}^{128}$	128
Classifier	BatchNorm1d	128 features	128
	Dropout	$p = 0.2$	128
	Linear	$128 \to 10$	10

[a] Operations are listed in execution order. Sequence outputs are expressed as (timesteps, features), with the batch dimension omitted. The LSTM and GRU variants share the listed configuration; the final output comprises unnormalized class logits.

3.2.5 Vision Transformer

The Vision Transformer represents the image as a sequence of embedded patches augmented with a learnable classification token and positional embeddings. This representation supports content-dependent interactions across patches through self-attention. Unlike convolution, self-attention does not intrinsically encode spatial locality or translation equivariance; positional embeddings supply information about token location.

Table 6 details the patch construction, token-sequence formation, encoder configuration, and classification head. It follows the processing sequence from image patches to class logits, specifying corresponding intermediate representations. The architecture contains $105,546$ trainable parameters.

Table 6: **Vision Transformer Architecture** on Fashion-MNIST.

Stage	Operation	Configuration	Output[a]
Embedding	Strided convolution	28×28 image; 4×4 patches; kernel 4, stride 4; $D = 64$	N patches
Tokenization	Flatten grid	Patch sequence	$(49, 64)$
Sequence	Prepend CLS token	$x_{\text{cls}} \in \mathbb{R}^{1\times 64}$	$Z_0 \in \mathbb{R}^{50\times 64}$
	Add position embeddings	$P \in \mathbb{R}^{1\times 50\times 64}$	$(50, 64)$
	Dropout	$p = 0.1$	$(50, 64)$
Encoder	Transformer Encoder	$L = 3$, $H = 4$ heads; MLP width 128; GELU; dropout $p = 0.1$	$(50, 64)$
Readout	Select CLS token	Encoded CLS token	64
	LayerNorm	64 features	64
	Linear	$64 \to 10$	10

[a] CLS denotes the classification token; the patch count is $N = (28/4)^2 = 49$. Operations are listed in execution order. Token-sequence outputs are expressed as (tokens, embedding dimensions), omitting the batch dimension. The positional-embedding shape retains its broadcast dimension. The final output comprises unnormalized class logits.

For an input sequence Z, each attention head evaluates scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad d_k = \frac{D}{H} = 16 \tag{3}$$

Here, Q, K, and V denote queries, keys, and values. Query–key similarities determine normalized attention weights, which aggregate value representations into contextual token outputs.

4 Experimental Protocol

4.1 Training Configuration and Checkpoint Selection

All architectures use Adam (lr $= 10^{-3}$, $\beta_1 = 0.9, \beta_2 = 0.999$) with batch size $B = 64$ for 10 epochs. Each epoch comprises training followed by validation, averaging sample-weighted cross-entropy over each partition. Checkpoints update only when validation Macro-F1 strictly improves; ties retain the earlier checkpoint. Validation uses $10,000$ Fashion-MNIST samples; Table 1 gives the CIFAR-10 size. The selected checkpoint is restored before test evaluation.

Cross-entropy supplies the common training objective. Accuracy is the proportion of correct predictions, and Macro-F1 is the unweighted mean of the class-wise F1 scores. These predictive criteria are considered jointly with parameter count and execution time. All scheduled epochs are completed without learning-rate scheduling, early stopping, automatic mixed precision, or gradient scaling. Architecture settings are fixed rather than selected through an automated search. The extent of any prior manual tuning is not documented.

4.2 Experimental Design

Four experiments examine the relationship between architectural inductive bias, parameter allocation, robustness, and dataset characteristics:

1. **Experiment 1 (Main Architecture Comparison):** This experiment tests the hypothesis that CNN attains the highest accuracy and Macro-F1 through 2D spatial locality, while MLP/CNN provide faster inference than sequential RNNs. Architecture family and input representation are varied; the Fashion-MNIST partition (seed = 42), Adam optimizer (lr = 10^{-3}), batch size 64, and 10 epochs remain fixed.
2. **Experiment 2 (Matched Parameter Budget Study):** This experiment tests whether the advantage of convolutional inductive bias persists when the architectures are constrained to $\approx$ 105K parameters, relative to 2D inputs represented as flattened vectors or row sequences. Architecture is varied under a fixed parameter budget ($\approx$ 105K).
3. **Experiment 3 (Noise & Occlusion Robustness):** This experiment tests the hypothesis that global and recurrent representations exhibit different degradation patterns under high-frequency Gaussian noise ($\sigma \in [0, 0.5]$) and localized 8×8 center cutout occlusion.
4. **Experiment 4 (CIFAR-10 Generalization Extension):** We examine whether the performance advantage of spatial architectures (CNN/ViT) over flattened representations (Linear/MLP) becomes more pronounced for 3-channel natural images with background clutter.

Architecture comparisons vary model family and input representation while sharing dataset partitions, optimizer, training schedule, batch size, loss, and evaluation functions. Matched-budget experiments reuse Fashion-MNIST loaders and the training function, with hidden widths of 124 (MLP), 88 (LSTM), and 102 (GRU); CNN uses base width 12 and fully connected width 86. ViT retains its methodology configuration. CIFAR-10 follows the same protocol with dataset-specific channels, image dimensions, and split.

The robustness study reuses the trained models. Noise levels $\sigma \in \{0.0, 0.1, 0.2, 0.3, 0.5\}$ are defined in normalized input space; Gaussian perturbations are followed by clipping to $[-1.0, 1.0]$. Center cutout replaces an 8×8 region with -1.0. Accuracy and Macro-F1 are recomputed on the perturbed test set. Gaussian samples are drawn afresh during each model evaluation, so the corruption levels are shared but the noise realizations are not paired across architectures. Interpretability uses learned Linear weight templates and the absolute input gradient of the true-class logit for the first image of the first test batch.

The conclusion criteria are descriptive differences in Accuracy and Macro-F1 considered jointly with parameter count and execution time. The matched-budget comparison tests whether the architecture ranking persists at similar capacity; the corruption study compares degradation across noise levels and occlusion; the CIFAR-10 extension examines the change in relative predictive performance between datasets. These comparisons do not constitute tests of statistical significance.

The aggregate tables were synchronized with the repository CSV summaries. Run identifiers and checkpoint hashes are unavailable.

4.3 Hardware and Software Environment

Experiments used WSL2 (Linux) on Windows 11, using an Intel Core Ultra 9 275HX CPU and an NVIDIA GeForce RTX 5060 Laptop GPU with 32 GB RAM. The software environment comprised Python 3.13.13 and PyTorch 2.12.1 (CUDA 13.0 build, +cu130), with deterministic CuDNN flags enabled (seed = 42). All reported training runs were executed on the GPU.

Project documentation corroborates the reported CPU, GPU, RAM, and WSL2 environment. The exact Python and PyTorch versions remain author-reported because a captured environment record is unavailable. The declared minimum dependencies are PyTorch 2.0.0, Torchvision 0.15.0, NumPy 1.21.0, Matplotlib 3.5.0, and scikit-learn 1.0.0; they do not constitute a locked environment. Fixed pseudorandom seeds therefore do not establish identical results across library versions and hardware.

4.4 Reproducibility and Measurement Protocol

The protocol below outlines experimental reconstruction, checkpoint selection, and timing measurement, together with the available verification records and their limitations:

1. **Reconstruction and provenance:** An archived implementation and reconstruction instructions specify dataset acquisition, preprocessing, architecture configuration, training, checkpoint restoration, evaluation, and the extension studies. The archived revision identifies the implementation reviewed for this report, rather than an independently verified generating revision for every tabulated result. Stored model states are not included in that archive; reconstruction is supported through the complete training and evaluation procedure. Operational commands and artifact locations are provided separately in the accompanying technical documentation.
2. **Checkpoint selection and validation metrics:** Maximum validation accuracy and maximum validation Macro-F1 are aggregated independently, whereas checkpoint selection follows Macro-F1 alone. Consequently, both maxima need not correspond to the selected checkpoint epoch. Test metrics use the checkpoint selected by validation Macro-F1; checkpoint-specific validation accuracy requires the corresponding training history. Per-epoch statistics are computed and visualized, yet persistent histories and result-to-revision mappings are unavailable in the archived materials.
3. **Training-time measurement:** Training duration is accumulated over training passes and excludes validation.
4. **Test-time measurement:** Test duration spans the complete test-set traversal, including data loading, device transfers, prediction, transfer of outputs to the CPU, and collection of example images, followed by device synchronization. It therefore measures an end-to-end test pass rather than forward-pass latency per image. No explicit inference warm-up or repeated timing average is used.
5. **Documentation and verification:** The project's detailed AI-use record identifies named users, representative prompts, affected components, verification narratives, and responsible members. These are author-declared records; the reported model-training and code-verification procedures have not been independently repeated during this report revision.

5 Results and Discussion

5.1 Training Dynamics

Learning dynamics provide context for the final predictive results. The training and validation trajectories in Figure 4 trace model behavior over the reported 10-epoch schedule.

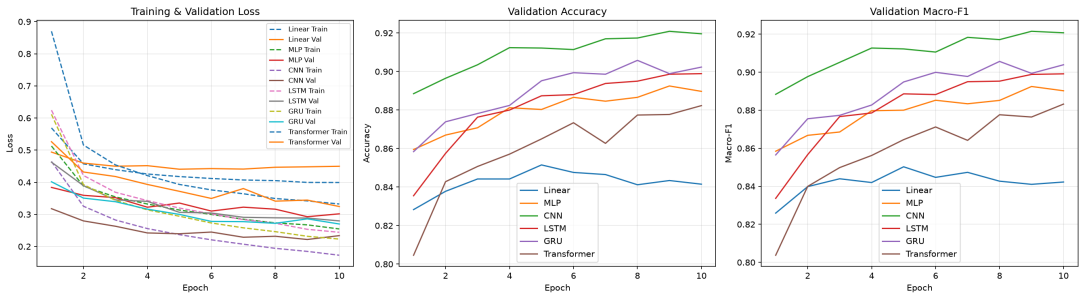


Figure 4: Training and validation loss, accuracy, and Macro-F1 curves across 10 epochs on Fashion-MNIST.

CNN maintains the strongest validation metrics, while Linear initially improves before plateauing. Falling training loss with inconsistent validation gains suggests generalization gaps. The curves cannot isolate architectural bias from optimization or training duration. Missing epoch histories prevent exact loss and checkpoint-epoch reconstruction; longer schedules require separate evaluation.

5.2 Fashion-MNIST Benchmark

Predictive performance and computational cost on Fashion-MNIST are compared in Table 7.

Table 7: **Main Quantitative Comparison on Fashion-MNIST** (10 Epochs, Seed = 42).

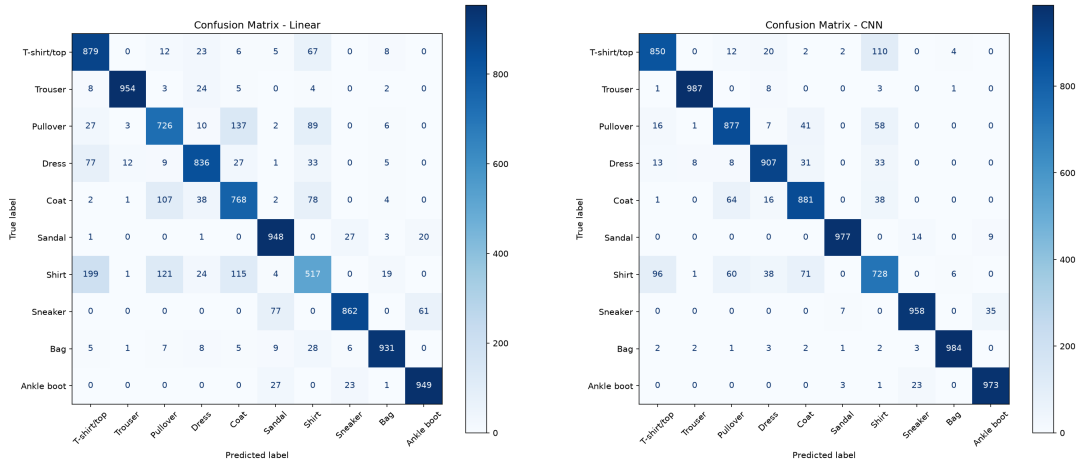
Metric[a]	Linear	MLP	CNN	LSTM	GRU	Transformer
↓ Params	**7,850**	235,658	207,018	214,538	161,290	105,546
↑ Val Acc	0.8515	0.8925	**0.9209**	0.8989	0.9058	0.8823
↑ Val F1	0.8503	0.8925	**0.9215**	0.8992	0.9057	0.8832
↑ Test Acc	0.8370	0.8855	**0.9122**	0.8922	0.8966	0.8815
↑ Test F1	0.8353	0.8846	**0.9120**	0.8919	0.8957	0.8818
↓ Train (s)	**27.41**	30.17	34.70	34.94	33.57	53.78
↓ Infer (s)	**0.42**	0.45	0.50	0.53	0.49	0.56

[a] Validation accuracy and Macro-F1 are independent maxima across epochs and may occur at different epochs. Test metrics use the checkpoint selected by validation Macro-F1. Train time is cumulative training-pass wall-clock time on the GPU and excludes validation. Arrows indicate preferred directions; bold marks the best reported value in each row.

CNN achieves the highest predictive performance (91.22% Test Accuracy, 0.9120 Test Macro-F1) with 207,018 parameters, fewer than either MLP (235,658) or LSTM (214,538). Parameter count describes model size rather than measured execution cost, so this comparison does not imply that CNN is also the fastest architecture. The single-seed results likewise do not establish statistical significance. This comparison motivates a closer examination of the relationship between spatial inductive bias and parameter efficiency.

5.2.1 Qualitative Results and Confusion Matrices

Aggregate metrics summarize overall performance but offer limited insight into the distribution of errors across classes. The test-set confusion matrices for the Linear baseline and CNN in Figure 5 provide this class-level view, showing where predictions agree with the true labels and where errors are concentrated. Each row corresponds to a true class and each column to a predicted class. Diagonal entries record correct predictions, while off-diagonal entries identify the destination of each misclassification.



(a) Linear Classifier Confusion Matrix (b) CNN Classifier Confusion Matrix

Figure 5: Comparison of test set confusion matrices between the Linear baseline and CNN.

Both matrices show off-diagonal errors among visually similar upper-body garments, including Shirt, T-shirt/top, Pullover, and Coat. The stronger aggregate CNN performance therefore coexists with persistent ambiguity between related categories. These class-level errors qualify the overall ranking: improved aggregate performance does not imply uniform separation of every category. Similar appearances provide a plausible explanation for the observed garment confusions, but the matrices alone cannot determine which visual features caused individual errors. These patterns motivate the subsequent error analysis rather than establishing a specific causal mechanism.

At the level of individual images, correct and incorrect CNN predictions offer a complementary view of these classification ambiguities. Ground-truth and predicted labels accompany the examples in Figure 6, permitting direct inspection of individual successes and failures.

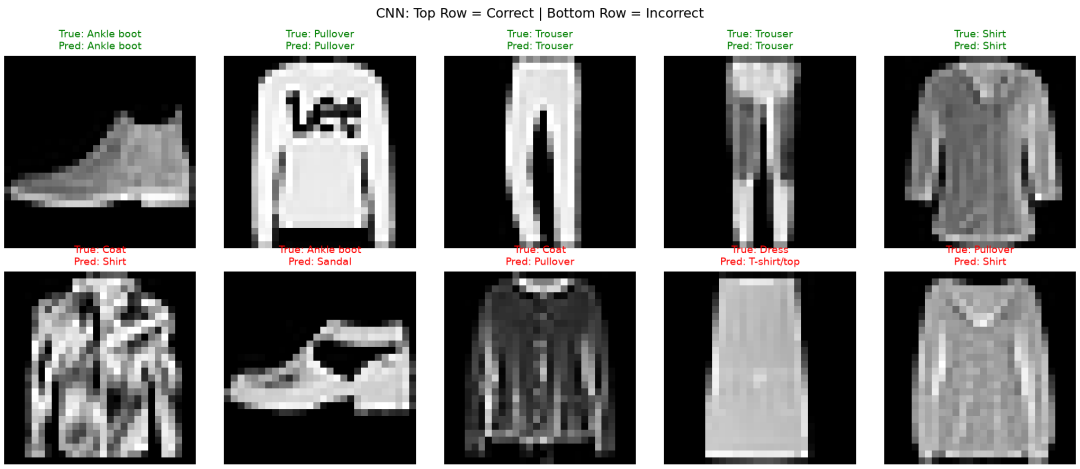


Figure 6: Qualitative evaluation of CNN predictions: Correct predictions (top row) vs. Misclassified samples (bottom row).

The displayed errors include Coat predicted as Shirt, Pullover predicted as Shirt, and Ankle boot predicted as Sandal. These examples illustrate the garment and footwear ambiguities visible in the confusion matrices. They provide qualitative context for the aggregate results, but the selected examples do not quantify the frequency of each error type across the full test set.

5.2.2 Architectural Trade-offs

The architectures are compared in terms of representation, predictive performance, parameter efficiency, and computational cost, extending the analysis beyond accuracy alone.

The contrast between 2D spatial locality and flattened representations is evident when comparing CNN with Linear/MLP. Flattening a 28×28 image removes the explicit encoding of 2D topology: vertically adjacent pixels (r, c) and $(r+1, c)$ are separated by 28 indices in $\mathbb{R}^{784}$. Nonlinear feature combinations allow MLP to improve over the Linear classifier by +4.85 points of Test Accuracy, but its first layer alone requires $784 \times 256 = 200,704$ weights. By exploiting local 3×3 receptive fields and shared weights, CNN exceeds MLP by +2.74 points of Test Macro-F1 (+2.67 points of accuracy) while using 12% fewer parameters.

For the recurrent architectures, the row-sequential inductive bias provides a complementary basis for comparing LSTM and GRU. Representing each image as 28 row timesteps encodes the vertical progression of garment silhouettes, such as collar $\rightarrow$ sleeves $\rightarrow$ hem. LSTM and GRU attain 89.22% and 89.66% Test Accuracy, respectively; GRU leads by 0.44 points in this single-seed comparison. GRU also uses 24.8% fewer parameters (161.3K vs. 214.5K) and trains 3.9% faster (33.57 s vs. 34.94 s). The relative performance is compatible with the hypothesis that simplified GRU gating is adequate for the short 28-step sequence; this comparison does not establish the gating mechanism as the cause of the observed difference.

Patch self-attention yields a different trade-off in the Vision Transformer. With $105,546$ parameters, the Patch ViT achieves 88.15% Test Accuracy. Unlike convolution, self-attention does not intrinsically encode spatial locality or translation equivariance; spatial relationships must

instead be learned from data using positional encodings. This distinction is consistent with the inductive-bias explanation for its lower accuracy than CNN when trained from scratch on 50,000 small images, but the experiment does not isolate this mechanism from optimization and training-duration effects.

The inference-speed hypothesis is only partly supported by the synchronized table: MLP is faster than both recurrent models, while CNN is slightly slower than GRU but faster than LSTM. The predictive-performance hypothesis is supported by the reported CNN ranking. These timing relationships are specific to the reported measurements and do not establish a general speed ranking.

5.3 Matched Parameter Budget Comparison

A comparison at similar parameter budgets helps assess whether CNN's advantage persists when model size is constrained. The nonlinear architecture families in Table 8 are resized toward $\approx$ 105K parameters; their realized counts remain close to this target, ranging from roughly 104K to 106K.

Table 8: **Matched Parameter Budget Comparison on Fashion-MNIST**.

Metric[a]	MLP_105k	CNN_105k	LSTM_105k	GRU_105k	ViT_105k
↓ Params	105,968	104,900	105,258	**104,662**	105,546
↑ Val Acc	0.8871	**0.9155**	0.8975	0.9021	0.8831
↑ Val F1[b]	0.8865	**0.9164**	0.8976	0.9023	0.8827
↑ Test Acc	0.8791	**0.9076**	0.8935	0.8957	0.8822
↑ Test F1	0.8786	**0.9080**	0.8931	0.8954	0.8813
↓ Train (s)	33.19	**32.26**	33.02	32.46	52.58

[a] Validation accuracy and Macro-F1 are independent maxima across epochs and may occur at different epochs. Test metrics use the checkpoint selected by validation Macro-F1. Train time is cumulative training-pass wall-clock time on the GPU and excludes validation. Arrows indicate preferred directions; bold marks the best reported value in each row.

[b] Validation Macro-F1 values have been checked against the current repository CSV summary.

`CNN_105k` attains 0.9164 validation Macro-F1 and 91.55% validation accuracy. Its reported Macro-F1 exceeds `GRU_105k` (0.9023), `LSTM_105k` (0.8976), `MLP_105k` (0.8865), and `ViT_105k` (0.8827). Its validation Macro-F1 is 0.51 percentage points below the full-size CNN (0.9215). All validation Macro-F1 entries agree with the current repository CSV summary.

5.4 CIFAR-10 Extension Benchmark

Extending the comparison to CIFAR-10 ($3 \times 32 \times 32$) examines variation in object position and background color beyond Fashion-MNIST's centered objects and uniform black backgrounds. Table 9 summarizes performance in this setting.

Table 9: **Extension Benchmark on CIFAR-10** ($3 \times 32 \times 32$, 10 Epochs, Seed = 42).

Metric[a]	Linear	MLP	CNN	LSTM	GRU	Transformer
↓ Params	**30,730**	821,386	268,746	249,354	187,402	108,554
↑ Val Acc	0.3858	0.5468	**0.7036**	0.5982	0.6220	0.6642
↑ Val F1	0.3854	0.5453	**0.6999**	0.5966	0.6196	0.6599
↑ Test Acc	0.3830	0.5479	**0.6997**	0.6060	0.6179	0.6628
↑ Test F1	0.3827	0.5456	**0.6968**	0.6053	0.6166	0.6601
↓ Train (s)	**29.82**	32.50	33.60	36.04	34.43	60.03
↓ Infer (s)	0.56	0.55	**0.54**	0.60	0.57	0.69

[a] Validation accuracy and Macro-F1 are independent maxima across epochs and may occur at different epochs. Test metrics use the checkpoint selected by validation Macro-F1. Train time is cumulative training-pass wall-clock time on the GPU and excludes validation. Arrows indicate preferred directions; bold marks the best reported value in each row.

Flattened representations consequently achieve substantially lower maximum validation accuracy: Linear attains 38.58% and MLP 54.68%, compared with 70.36% for CNN, a +15.68-point advantage over MLP. The Transformer reaches 66.42%, exceeding MLP by 11.74 points and reversing their ordering on Fashion-MNIST. The recurrent architectures occupy an intermediate position (GRU 62.20%, LSTM 59.82%). These findings are consistent with the hypothesis examined in Experiment 4.

5.5 Robustness and Interpretability

The Gaussian-noise curves in Figure 7 track test accuracy as corruption increases.

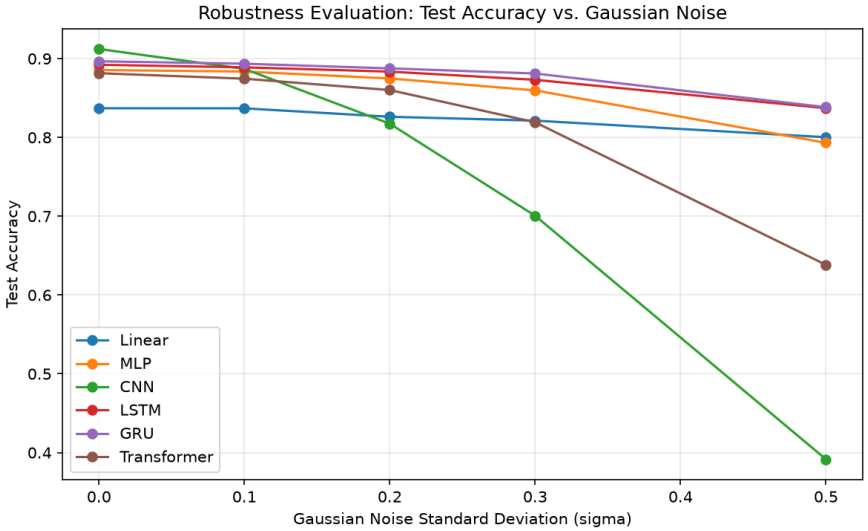


Figure 7: Test accuracy under Gaussian noise of standard deviation σ.

Under Gaussian noise ($\sigma = 0.5$), CNN with local 3×3 filters attains 39.15% Test Accuracy, below LSTM (83.71%) and Linear (80.02%). With 8×8 center cutout, CNN is most robust (80.81%, versus 65–75% for the others), consistent with locality and weight sharing. These contrasting rankings indicate corruption-dependent robustness: CNN's advantage under localized occlusion does not extend to strong Gaussian perturbations.

Input-gradient saliency complements corruption testing with a qualitative comparison of model sensitivity to a shared test example (Figure 8).

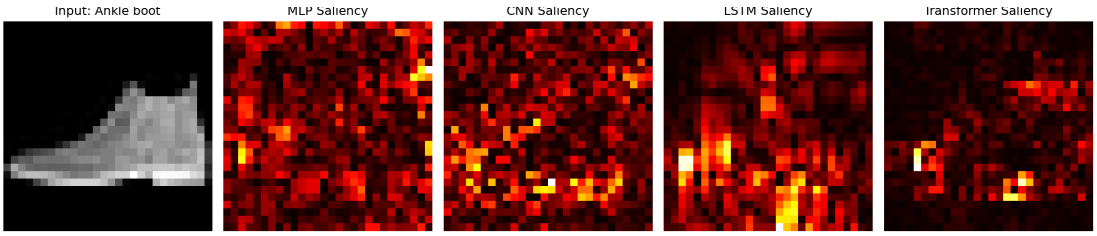


Figure 8: Input-gradient saliency maps for a shared Fashion-MNIST test example.

The ankle-boot example illustrates model-dependent input-gradient sensitivity. LSTM recurrent layers temporarily use training mode for gradient calculation while the remaining network stays in evaluation mode. Active recurrent dropout can make saliency nondeterministic; no repeated analysis is supplied. These maps establish neither general neckline or sleeve preferences nor causal feature use.

5.6 Error Analysis

The confusion matrices (Figure 5) show garment and footwear confusions; a recurrent-memory explanation remains an untested hypothesis:

1. **Garment ambiguity:** *Shirt* (Class 6), *T-shirt/top*, *Pullover*, and *Coat* remain confused. At 28×28 resolution, buttons, collars, and sleeve seams may blur into 1-pixel-wide structures; this explanation is untested. Figure 6 includes Coat and Pullover predicted as Shirt.

2. **Footwear confusion:** Ankle boots and sneakers are confused, and Figure 6 includes an ankle boot predicted as Sandal. Reduced boot-shaft contrast against the background is a possible cause, not isolated by these predictions.

3. **Recurrent-memory hypothesis:** LSTM/GRU may lose upper-image features, including *T-shirt/top* and *Pullover* necklines at rows $t = 2 \dots 6$, as lower rows ($t = 20 \dots 28$) influence h_{28}. Classification uses this terminal representation, so the proposed mechanism concerns retention of earlier rows. No timestep-level attribution or controlled memory experiment substantiates this hypothesis. Bidirectional recurrence or attention pooling over $h_{1:28}$ are proposed comparisons, not demonstrated improvements.

Higher-resolution inputs could test the garment and footwear hypotheses; these potential improvements remain unverified.

5.7 Limitations

The experiments use a single random seed (42) and a fixed 10-epoch training schedule, without data augmentation or cosine learning rate warmup. The experiments do not establish whether longer schedules or augmentation would alter the ranking, particularly for the Vision Transformer.

The single-seed design precludes statistical-significance claims. Splits, preprocessing, architectures, the fixed schedule, checkpoint policy, corruption operators, and reconstruction are specified; downloaded archive footprint, environment locks, checkpoint hashes, epoch logs, and links between results and commits remain unavailable. Aggregate tables match the available CSV summaries. Checkpoint-specific validation accuracy requires the missing histories. Error mechanisms remain hypotheses, and qualitative saliency observations do not establish causality.

6 Conclusion

In the synchronized Fashion-MNIST and CIFAR-10 tables, CNN attains the highest predictive accuracy (91.22% test accuracy on Fashion-MNIST, 70.36% validation accuracy on CIFAR-10) and Macro-F1 among the evaluated architectures. Its parameter count is lower than those of MLP and LSTM, but its predictive advantage does not imply dominance in every resource measure: the Linear and MLP models have shorter reported inference times, and GRU and the Transformer use fewer parameters. GRU uses fewer parameters and has shorter reported training and inference times than LSTM. The results are consistent with an advantage for spatial inductive bias under this protocol; broader causal or generalization claims require additional evidence.

AI Usage Disclosure

In accordance with Section 5 of the Course Project Handbook, our group discloses the following author-declared uses of generative AI tools during Assignment 1 (representative prompts and author-declared verification records are provided in the repository AI-use disclosure):

- **Tool:** Google Gemini (2026). **Used by:** All 4 group members across M1 and M2.
- **Scope of Assistance:** Subplot layout formatting in Matplotlib, CSV export boilerplate, and LaTeX table/TikZ typesetting.
- **Student Verification:**
  The authors report local training and verification of the Linear, MLP, CNN, LSTM/GRU, and Transformer architectures, training loops, and evaluation protocols against official PyTorch documentation (`seed=42`). Aggregate table entries were checked against the available CSV summaries. This consistency check does not independently reproduce the experiments.
- **Report Editing:** OpenAI Codex assisted with academic editing, figure and table presentation, CSV consistency checks, and PDF layout verification. No training or new experiments were performed during report editing.